

API SECURITY RISK ANALYSIS

July 30, 2026

Prepared by: Saad Ahmed

Executive Summary

This report presents a read-only API Security Risk Analysis conducted on a publicly accessible demonstration API as part of a cybersecurity security assessment. The objective of the assessment was to evaluate the API's security posture by reviewing its authentication mechanisms, authorization controls, response data, HTTP headers, input handling, and other common API security practices without performing any unauthorized or intrusive testing.

The assessment was performed using industry-standard tools, including Postman and browser developer tools, following ethical testing practices and the principles outlined in the OWASP API Security Top 10. All testing was limited to publicly available endpoints and documentation-based analysis, ensuring that no exploitation, denial-of-service testing, or attempts to bypass security controls were conducted.

During the assessment, the API was examined for common security risks such as unauthenticated endpoints, excessive data exposure, insufficient authentication and authorization controls, missing rate limiting, information disclosure through HTTP headers, and input validation weaknesses. Each identified observation was evaluated based on its potential security impact and assigned an appropriate risk severity. Where applicable, practical remediation recommendations have been provided to help strengthen the overall security posture of the API.

The purpose of this report is to demonstrate a structured API security assessment methodology and provide actionable recommendations that align with modern application security best practices. While the analyzed API is intended for testing and educational purposes, the assessment methodology and recommendations reflect approaches commonly used by security analysts and application security consultants when evaluating production APIs.

Scope of Assessment

The scope of this assessment was limited to a read-only security analysis of the **JSON Placeholder** public REST API, a freely available demonstration API designed for testing and learning purposes. The assessment focused on evaluating the API's security posture through documentation review, request and response analysis, and observation of common API security controls without performing any intrusive or unauthorized testing.

The assessment included the following activities:

- Reviewing the API documentation and available endpoints.
- Testing API endpoints using **Postman** and **Insomnia**.

- Inspecting HTTP requests, responses, headers, and payloads using **Browser Developer Tools**.
- Evaluating authentication and authorization mechanisms.
- Assessing API responses for excessive data exposure.
- Reviewing HTTP security headers and response behavior.
- Observing input handling and error responses.
- Identifying potential security risks based on the **OWASP API Security Top 10** and general API security best practices.

The assessment was conducted under the following constraints:

- Only publicly accessible endpoints provided by the JSON Placeholder API were tested.
- Testing was limited to safe, read-only requests and documentation-based analysis.
- No attempts were made to exploit vulnerabilities, bypass authentication or authorization controls, modify server-side data, perform denial-of-service testing, or conduct any activity that could negatively impact the availability or integrity of the service.
- All observations and recommendations presented in this report are based solely on the publicly accessible behavior of the API within the defined assessment scope.

Target API: JSON Placeholder Public REST API

Assessment Tools:

- Postman
- Insomnia
- Browser Developer Tools (Network Inspector)

Assessment Methodology: Read-only API Security Risk Analysis based on the OWASP API Security Top 10 and industry-standard API security assessment practices.

Methodology

The API Security Risk Analysis was conducted using a structured, read-only assessment methodology based on the **OWASP API Security Top 10** and established API security assessment practices. The objective was to identify potential security risks through observation and analysis without performing any unauthorized or intrusive testing.

The assessment followed the methodology outlined below:

1. API Documentation Review

The publicly available JSONPlaceholder API documentation was reviewed to understand the available endpoints, supported HTTP methods, expected request parameters, and response formats. This provided the foundation for planning the assessment.

2. Endpoint Enumeration

The available API endpoints were identified and cataloged. Each endpoint was reviewed to understand its intended functionality and determine whether it exposed publicly accessible resources.

3. Request and Response Analysis

API requests were performed using **Postman** and **Insomnia**. Responses were analyzed to evaluate:

- HTTP status codes
- Response headers
- JSON response bodies
- Response structure and exposed data
- Response consistency across endpoints

4. HTTP Header Inspection

Browser Developer Tools and API testing tools were used to inspect HTTP request and response headers. The assessment focused on identifying exposed server information, content types, caching behavior, and the presence or absence of common HTTP security headers.

5. Authentication and Authorization Assessment

The API was evaluated to determine whether authentication mechanisms were implemented and whether access to resources required authorization. The assessment focused on identifying publicly accessible endpoints and reviewing access control behavior without attempting to bypass any security controls.

6. Input Handling Observation

API endpoints that accepted parameters were tested using valid and unexpected input values to observe how the API handled user input, validation, and error responses. Testing remained within safe and ethical boundaries.

7. Security Risk Identification

Observed behaviors were compared against common API security risks, including those described in the OWASP API Security Top 10. Potential issues such as unauthenticated access, excessive data exposure, missing security headers, inadequate input validation, and the absence of rate limiting were documented where applicable.

8. Risk Assessment and Recommendations

Each identified finding was evaluated based on its potential business impact and assigned an appropriate risk severity (Low, Medium, or High). Practical remediation recommendations were provided to help improve the overall security posture of the API.

Throughout the assessment, all testing activities were conducted in accordance with ethical security testing practices. No exploitation attempts, denial-of-service testing, authentication bypasses, or modifications to server-side resources were performed.

Finding 1 – Unauthenticated Access to Public API Endpoints

Field	Details
Finding ID	F-01
Finding Title	Unauthenticated Access to Public API Endpoints
Risk Category	Authentication / Access Control
Risk Severity	Medium (<i>Informational for this demo API; Medium if observed in a production API exposing sensitive resources.</i>)
Affected Endpoints	GET /usersGET /posts?userId=1
OWASP API Security Mapping	API2:2023 – Broken Authentication
Description	The assessed endpoints were accessible without requiring any form of authentication, such as an API key, bearer token, or session-based authentication. Requests sent through Postman and Insomnia returned successful responses

Field	Details
	(HTTP 200 OK) containing the requested data without verifying the identity of the requester.
Evidence	Requests to <code>GET /users</code> and <code>GET /posts?userId=1</code> were executed without authentication credentials and returned valid JSON responses. Screenshots of the requests and responses are included in the Evidence section of this report.
Potential Security Impact	If this behavior were present in a production API containing sensitive or customer-specific information, unauthorized users could access data without identity verification, potentially leading to information disclosure, privacy violations, and non-compliance with security or regulatory requirements.
Business Impact	Unrestricted access to sensitive API resources may expose customer information, reduce user trust, increase the likelihood of data breaches, and result in reputational damage or regulatory penalties.
Likelihood	Medium
Recommendation	Implement appropriate authentication mechanisms such as OAuth 2.0, JWT Bearer Tokens, or API Keys for endpoints that expose sensitive information. Enforce authentication before processing requests and apply the principle of least privilege to ensure users can only access resources they are authorized to view.
Assessment Notes	JSON Placeholder is intentionally designed as a public demonstration API for development and testing purposes. Therefore, unauthenticated access is expected behavior in this environment. This observation is documented to illustrate the security implications if similar behavior were identified in a production API.

Finding 2 – Excessive Data Exposure in API Responses

Field	Details
Finding ID	F-02
Finding Title	Excessive Data Exposure in API Responses
Risk Category	Data Exposure / Information Disclosure
Risk Severity	Medium (<i>Informational for this demo API; Medium if observed in a production API.</i>)
Affected Endpoints	<code>GET /users</code> <code>GET /users/1</code>
OWASP API Security Mapping	API3:2023 – Broken Object Property Level Authorization
Description	The API responses contained a comprehensive set of user-related properties, including personal and organizational information such as email address, phone number, website, company details, and address information. While appropriate for a public demonstration API, returning more information than is required by the client application can lead to excessive data exposure in production environments.
Evidence	Requests to <code>GET /users</code> and <code>GET /users/1</code> returned complete user objects containing multiple fields beyond basic identification, including nested address and company information. Screenshots of the API responses are included in the Evidence section of this report.
Potential Security Impact	Exposing unnecessary data increases the amount of information available to unauthorized or unintended users. If sensitive or personally identifiable information (PII) is unnecessarily included in API responses, it may facilitate information gathering, privacy violations, or support further attacks against users or the organization.
Business Impact	Excessive data exposure may increase regulatory compliance risks, compromise user privacy, and expand the attack surface by revealing information that is not required by the client application. Limiting data exposure also improves data governance and reduces the impact of potential security incidents.
Likelihood	Medium

Field	Details
Recommendation	Return only the minimum set of data required by the client application by implementing the principle of least privilege for API responses. Use response filtering, Data Transfer Objects (DTOs), or field-level access controls to prevent unnecessary properties from being included in API responses. Review API responses regularly to ensure sensitive information is not unintentionally exposed.
Assessment Notes	JSONPlaceholder is intentionally designed to provide complete sample datasets for testing and development purposes. Therefore, the exposed information is expected within the context of this demonstration API. This observation highlights the potential security implications if similar response behavior were identified in a production environment containing real user data.

Finding 3 – Potential Authorization Control Weakness

Field	Details
Finding ID	F-03
Finding Title	Potential Authorization Control Weakness
Risk Category	Authorization / Access Control
Risk Severity	Medium (<i>Informational for this demo API; High if observed in a production API managing user-specific data.</i>)
Affected Endpoints	GET /posts?userId=1GET /posts?userId=2
OWASP API Security Mapping	API1:2023 – Broken Object Level Authorization (BOLA)
Description	During the assessment, the <code>userId</code> query parameter could be modified to retrieve data associated with different users. The API did not enforce any authorization checks because it is a public demonstration API. In a production

Field	Details
	environment, allowing users to manipulate object identifiers without validating ownership could result in unauthorized access to other users' data.
Evidence	Requests were submitted to <code>GET /posts?userId=1</code> and subsequently modified to <code>GET /posts?userId=2</code> . Both requests returned successful (HTTP 200 OK) responses containing data for the specified user identifier without requiring authentication or authorization. Screenshots of the requests and responses are included in the Evidence section of this report.
Potential Security Impact	If implemented in a production environment without proper authorization controls, attackers could enumerate object identifiers and access resources belonging to other users. This could result in unauthorized disclosure of sensitive information and compromise data confidentiality.
Business Impact	Broken object-level authorization is one of the most common API security issues and may lead to unauthorized access to customer records, privacy violations, regulatory non-compliance, reputational damage, and loss of customer trust.
Likelihood	Medium
Recommendation	Implement server-side authorization checks for every request involving user-controlled object identifiers. Validate that the authenticated user is authorized to access the requested resource before returning any data. Apply object-level authorization consistently across all API endpoints and avoid relying solely on client-supplied identifiers such as <code>userId</code> .
Assessment Notes	JSONPlaceholder is intentionally designed as a public demonstration API and does not implement authentication or user-specific authorization. Therefore, modifying the <code>userId</code> query parameter is expected behavior in this environment. This observation highlights the potential risk if similar behavior were present in a production API handling real user accounts and sensitive data.

Finding 4 – Information Disclosure via HTTP Response Headers

Field	Details
Finding ID	F-04
Finding Title	Information Disclosure via HTTP Response Headers
Risk Category	Information Disclosure / Security Misconfiguration
Risk Severity	Low
Affected Endpoints	GET /usersGET /posts?userId=1
OWASP API Security Mapping	API8:2023 – Security Misconfiguration
Description	During the assessment, the HTTP response headers exposed information about the API's response behavior and underlying implementation. While no sensitive credentials or confidential information were disclosed, exposing unnecessary header information may provide attackers with additional context during the reconnaissance phase of an attack.
Evidence	Response headers were inspected using Browser Developer Tools, Postman, and Insomnia. Headers such as <code>Content-Type</code>, <code>Cache-Control</code>, <code>ETag</code>, and other response metadata were observed. Any server-specific or framework-related headers present have been documented in the accompanying evidence screenshots.
Potential Security Impact	Information disclosed through HTTP headers may help attackers identify the technologies, frameworks, or server configurations used by an application. Although the risk is generally low in isolation, such information can support reconnaissance activities and aid in planning targeted attacks when combined with other vulnerabilities.
Business Impact	Unnecessary disclosure of implementation details may increase the organization's exposure to reconnaissance-based attacks. While unlikely to result in a direct compromise, minimizing information disclosure follows the principle of reducing the attack surface and strengthens the overall security posture of the application.
Likelihood	Low

Field	Details
Recommendation	Review HTTP response headers and remove or suppress any unnecessary server or framework identification headers. Configure the web server or API gateway to expose only essential headers required for application functionality. Periodically review response headers as part of secure configuration management and security assessments.
Assessment Notes	The JSONPlaceholder API is a public demonstration service intended for development and testing. The observed response headers are not considered a vulnerability within the context of this assessment. This finding is documented to highlight the potential security implications of unnecessary information disclosure if similar behavior were observed in a production environment.

Conclusion

This API Security Risk Analysis evaluated the security posture of the **JSON Placeholder** public demonstration API using a structured, read-only assessment methodology aligned with the **OWASP API Security Top 10** and industry best practices. The assessment focused on reviewing publicly accessible endpoints, analyzing HTTP requests and responses, inspecting response headers, and evaluating common API security controls without performing any intrusive or unauthorized testing.

The assessment identified several observations that represent common API security considerations, including unauthenticated access to public endpoints, excessive data exposure in API responses, potential authorization control weaknesses, and information disclosure through HTTP response headers. While these behaviors are expected within the context of a public demonstration API, they illustrate security risks that could have significant implications if present in a production environment handling sensitive business or customer data.

The findings presented in this report emphasize the importance of implementing strong authentication and authorization mechanisms, minimizing data exposure through API responses, validating access to user-controlled resources, and following secure configuration practices. Addressing these areas helps reduce the attack surface, protect sensitive information, and improve the overall resilience of API-driven applications.

Overall, this assessment demonstrates a systematic approach to API security analysis and highlights the value of proactive security reviews during the software development lifecycle. By applying secure design principles and regularly assessing APIs against recognized security standards, organizations can strengthen their security posture, reduce business risk, and build greater trust in their applications and services.

